

Week one DSA problem sheet solution

1)Contains Duplicate

```
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        long long n = nums.size();
        if(n==1){
            return false;
        }
        sort(nums.begin(),nums.end());
        for(int i=0;i<n-1;i++){
            if(nums[i]==nums[i+1]){
                return true;
            }
        }
        return false;
    }
};
```

explanation : duplicate lie next to each other in a sorted array

2)Registration System

```
#include <bits/stdc++.h>
#include <string>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> request(n);
    for (auto &x : request)
        cin >> x;
    map<string, int> db;
    for (auto x : request) {
        if (db[x] == 0) {
            cout << "OK" << "\n";
        } else {
            string response = x + to_string(db[x]);
            cout << response << "\n";
        }
        db[x]++;
    }
    return 0;
}
```

explanation: use frequency map to store the number of times each request has appeared and send a response by using it

3)Valid Anagram

```
class Solution {
public:
bool isAnagram(string s, string t) {
vector<int> freq_s(26);
vector<int> freq_t(26);
if (s.length() != t.length()) {
return false;
}
for (auto x : s) {
freq_s[x - 'a']++;
}
for (auto x : t) {
freq_t[x - 'a']++;
}
for (int i = 0; i < 26; i++) {
if (freq_s[i] != freq_t[i]) {
return false;
}
}
return true;
}
};
```

explanation: Anagrams in this question have a property that frequency of letters used in Anagrams are exactly equal and it is a necessary and sufficient condition

4)First unique character in a string

```
class Solution {
public:
int firstUniqChar(string s) {
int n = s.length();
vector<int>freq(26);
for(auto x:s){
freq[x-'a']++;
}
for(int i=0;i<n;i++){
if(freq[s[i]-'a']==1){
return i;
}
}
return -1;
}
};
```

explanation: create a frequency vector and then check which one of the unique letters appears first by iterating through the string

5)Group Anagrams

```
class Solution {
public:
vector<vector<string>> groupAnagrams(vector<string>& strs) {
map<string,vector<string>>myMap;
for(auto x:strs){
string sorted = x;
sort(sorted.begin(),sorted.end());
if(myMap.find(sorted)==myMap.end()){
myMap[sorted] = {x};
}else{
myMap[sorted].push_back(x);
}
}
vector<vector<string>>ans;
for(auto &[key,value]: myMap){
ans.push_back(value);
}
return ans;
}
};
```

explanation: every anagram of a string has the same lexicographically sorted anagram so maintain a vector of anagrams for each of these unique lexicographically sorted anagram as a map and then just return the vectors /value of the map after moving it into a vector of vectors

6)Single Number

```
class Solution {
public:
int singleNumber(vector<int>& nums) {
int ans = 0;
for(auto x:nums){
ans = ans^x;
}
return ans;
}
};
```

explanation : we know that $a^a = 0$ and $a^b^c = a^c^b$ so when we do the xor operation throughout the vector nums all numbers which appear 2 times will ultimately become 0 and then the one which appears once will remain

7)Majority Element

```
class Solution {
```

```

public:
int majorityElement(vector<int>& nums) {
int cnt = 0;
int maxEl = 0;
for(auto num:nums){
if(cnt ==0){
maxEl = num;
}
if(num==maxEl){
cnt++;
}else{
cnt--;
}
}
return maxEl;
}
};

```

explanation : since the majority element appears more than $n/2$ times the largest frequency after it possible is $n/2 - 1$ and hence if we maintain a cnt variable for an element in the array which cancels out increases with a decrease if the current element is not that element ,it is sure that the one left at the end of it is the majority element

8)Best Time to Buy and Sell Stock

```

class Solution {
public:
int maxProfit(vector<int>& prices) {
int n = prices.size();
vector<int>Max(n-1);
for(int i=0;i<n-1;i++){
if(i==0){
Max[i]=prices[n-1];
}else{
Max[i]=max(Max[i-1],prices[n-1-i]);
}
}
int ans = 0;
for(int i=0;i<n-1;i++){
int buyPrice = prices[i];
int sellPrice = Max[n-2-i];
ans = max(ans,sellPrice-buyPrice);
}
return ans;
}
};

```

explanation : the maximum profit possible is $\max(\text{price}[i], \text{max price after } i\text{th day})$

9)Rotate Array

```

class Solution {
public:
void rotate(vector<int>& nums, int k) {
k%=nums.size();
std::rotate(nums.begin(),nums.begin()+nums.size()-k,nums.end());
}
};

public:
void rotate(vector<int>& nums, int k) {
k%=nums.size();
std::rotate(nums.begin(),nums.begin()+nums.size()-k,nums.end());
}
};

```

explanation : use the std:: rotate method directly

10)Next Permutation

```

class Solution {
public:
void nextPermutation(vector<int>& nums) {
int n = nums.size();
int idx = -1;
for(int i=n-2;i>=0;--i){
if(nums[i]<nums[i+1]){
idx = i;
break;
}
}
if(idx== -1){
sort(nums.begin(),nums.end());
return;
}
for(int i=n-1;i>=0;--i){
if(nums[i]>nums[idx]){
int temp = nums[i];
nums[i]=nums[idx];
nums[idx]=temp;
break;
}
}
std::reverse(nums.begin()+idx+1,nums.end());
}
};

```

explanation : to find the next permutation find the last index i such that $nums[i] < nums[i+1]$, then find the last index j such that $nums[j] > nums[i]$, swap them and reverse the segment of nums from i+1 to end

11)Maximum SubArray

```

class Solution {

```

```

public:
int maxSubArray(vector<int>& nums) {
int n = nums.size();
vector<long long> prefix(n);
vector<long long> Min(n);
prefix[0] = nums[0];
Min[0] = nums[0];
for (int i = 1; i < n; i++) {
prefix[i] = prefix[i - 1] + nums[i];
}
for (int i = 1; i < n; i++) {
Min[i] = min(Min[i - 1], prefix[i]);
}
long long ans = LLONG_MIN;
for (int i = 0; i < n; i++) {
if (i == 0) {
ans = max(ans, prefix[i]);
} else {
ans = max(ans, prefix[i]);
ans = max(ans, prefix[i] - Min[i - 1]);
}
}
return ans;
}
};

```

explanation : maximum sum sub array is nothing but $\max(\text{prefix}[i] - \min(\text{prefix}[j]))$ where $j < i$

12) Spiral Matrix

```

class Solution {
public:
vector<int> spiralOrder(vector<vector<int>>& matrix) {
int m = matrix.size();
int n = matrix[0].size();
vector<int> ans;
int top = 0, bottom = m - 1, left = 0, right = n - 1;
int cnt = 0;
while(left <= right || top <= bottom){
if(cnt == n * m){
break;
}
for(int i = left; i <= right; i++){
ans.push_back(matrix[top][i]);
cnt++;
}
top++;
if(cnt == n * m){
break;
}
for(int i = top; i <= bottom; i++){

```

```

ans.push_back(matrix[i][right]);
cnt++;
}
right--;
if(cnt==n*m){
break;
}
for(int i=right;i>=left;--i){
ans.push_back(matrix[bottom][i]);
cnt++;
}
bottom--;
if(cnt==n*m){
break;
}
for(int i=bottom;i>=top;--i){
ans.push_back(matrix[i][left]);
cnt++;
}
left++;
if(cnt==n*m){
break;
}
}
return ans;
}
};

```

explanation : since there is a pattern that the sequence always moves right,down,left,up ,we define boundaries and follow the pattern until all squares have been traversed through once

13)Search Insert Position

```

class Solution {
public:
int searchInsert(vector<int>& nums, int target) {
auto it = lower_bound(nums.begin(),nums.end(),target);
int idx = distance(nums.begin(),it);
return idx;
}
};

```

explanation : just find the lower_bound and the idx it points to and return it

14)First and Last Position of Element in Sorted Array

```

class Solution {
public:
vector<int> searchRange(vector<int>& nums, int target) {
vector<int>ans;

```

```

if(nums.empty()){
ans.push_back(-1);
ans.push_back(-1);
return ans;
}
auto it = lower_bound(nums.begin(),nums.end(),target);
if(it==nums.end()){
ans.push_back(-1);
ans.push_back(-1);
return ans;
}
int start = distance(nums.begin(),it);
if(nums[start]!=target){
ans.push_back(-1);
ans.push_back(-1);
return ans;
}
ans.push_back(start);
it = upper_bound(nums.begin(),nums.end(),target);
int end = distance(nums.begin(),it);
--end;
ans.push_back(end);
return ans;
}
};

```

explanation: just find the lower bound and upper bound indices ,l and u then the answer is {l,u-1}

15) Peak Index in a Mountain Array

```

class Solution {
public:
int peakIndexInMountainArray(vector<int>& arr) {
int n = arr.size();
int low = 0;
int high = n-2;
int ans = low;
while(low<=high){
int mid = (low+high)/2;
if(arr[mid]<arr[mid+1]){
low = mid+1;
ans = low;
}else if(arr[mid]>arr[mid+1]){
high = mid-1;
}
}
return ans;
}
};

```

explanation : binary search with the condition that arr[mid]<arr[mid+1]

16) Search in Rotated Sorted Array

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int n = nums.size();
        if (n == 0) return -1;
        int low = 0;
        int high = n - 1;
        while (low < high) {
            int mid = low + (high - low + 1) / 2;
            if (nums[mid] >= nums[0]) {
                low = mid;
            } else {
                high = mid - 1;
            }
        }
        int peak = low;

        auto start = nums.begin();
        if (target >= nums[0] && target <= nums[peak]) {
            auto it = lower_bound(start, start + peak + 1, target);
            if (it != start + peak + 1 && *it == target) {
                return distance(start, it);
            }
        } else {
            auto it = lower_bound(start + peak + 1, nums.end(), target);
            if (it != nums.end() && *it == target) {
                return distance(start, it);
            }
        }
        return -1;
    }
};
```

explanation : find the end of the rotation using binary search with condition $\text{nums}[\text{mid}] \geq \text{nums}[0]$ then split it into 2 parts and decide which part the target lies in and do normal binary search on that part

17) Koko Eating Bananas

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        int n = piles.size();
        long long low = 0;
        long long high = *max_element(piles.begin(), piles.end());
        long long ans = high;
        while (low <= high) {
            long long mid = (low + high) / 2;
            long long hours = 0;
            if (mid == 0) {
```

```

break;
}
for (int i = 0; i < n; i++) {
    if (piles[i] % mid == 0) {
        hours += (piles[i] / mid);
    } else {
        hours += (piles[i] / mid) + 1;
    }
}
if (hours <= h) {
    ans = min(ans, mid);
    high = mid - 1;
} else {
    low = mid + 1;
}
return ans;
};

```

explanation : do binary search on answer using the condition that he completes eating all piles

18) Aggressive Cows

```

#include <bits/stdc++.h> #include using namespace std;

#define fastio()
ios_base::sync_with_stdio(false);
cin.tie(NULL);
cout.tie(NULL) #define ll long long #define pb push_back #define mp make_pair #define nline "\n"

void solve() { int n, c; cin >> n >> c; vector x(n); for (auto &y : x) cin >> y; sort(x.begin(),
x.end()); vector dist(n - 1); vector pre(n - 1); for (int i = 0; i < n - 1; i++) { dist[i] = x[i + 1] -
x[i]; } pre[0] = dist[0]; for (int i = 1; i < n - 1; i++) { pre[i] = pre[i - 1] + dist[i]; } ll low =
*min_element(dist.begin(), dist.end()); ll high = x[n - 1] - x[0]; ll ans = low; while (low <= high) { ll
mid = (low + high) / 2; ll cnt = 1; ll lastPlaced = x[0];

for (int i = 1; i < n; i++) {
    if (x[i] - lastPlaced >= mid) {
        cnt++;
        lastPlaced = x[i];
    }
}
if (cnt >= c) {
    ans = max(ans, mid);
    low = mid + 1;
} else {
    high = mid - 1;
}

} cout << ans << "\n"; }

```

```
int main() { fastio(); int t; cin >> t; while (t--) { solve(); } return 0; }
```

explanation : binary search on answer using condition that we should be able to place all cows with the min distance between them as mid

19) Interesting drink

```
#include <bits/stdc++.h> #include using namespace std;
```

```
#define fastio()
```

```
ios_base::sync_with_stdio(false);
```

```
cin.tie(NULL);
```

```
cout.tie(NULL) #define ll long long #define pb push_back #define mp make_pair
```

```
void solve() { int n; cin >> n; vector x(n); for (auto &y : x) cin >> y; int q; cin >> q; vector m(q); for (auto &y : m) cin >> y; sort(x.begin(), x.end()); for (int i = 0; i < q; i++) { auto it = upper_bound(x.begin(), x.end(), m[i]); int idx = distance(x.begin(), it); cout << idx << "\n"; } }
```

```
int main() { fastio(); int t = 1; while (t--) { solve(); } return 0; }
```

explanation : sort the prices of the shops and for each day find the upper bound in that for the day's budget then the answer would be the previous one but since this is 0-indexed vector I can return the same one

20) Maximum Median

```
#include <bits/stdc++.h> #include using namespace std;
```

```
#define fastio()
```

```
ios_base::sync_with_stdio(false);
```

```
cin.tie(NULL);
```

```
cout.tie(NULL) #define ll long long #define pb push_back #define mp make_pair
```

```
void solve() { int n; long long k; cin >> n >> k; vector a(n); for (auto &x : a) cin >> x; sort(a.begin(), a.end()); long long low = a[n / 2]; long long ans = low; long long high = a[n - 1] + k; while (low <= high) { long long mid = (low + high) / 2; long long ops = 0; for (int i = n / 2; i < n; i++) { if (a[i] < mid) { ops += mid - a[i]; } } if (ops <= k) { ans = max(ans, mid); low = mid + 1; } else { high = mid - 1; } } cout << ans << "\n"; }
```

```
int main() { fastio(); int t = 1; while (t--) { solve(); } return 0; }
```

explanation : binary search on answer using the condition that the number of operations required for mid to be the answer is $\leq k$, to find the operation required is found by iterating through a from $n/2$ to n because before that there is no point in using the operation on them